

A Cooperative Computing Strategy for Blockchain-Secured Fog Computing

Di Wu^{ID}, *Student Member, IEEE*, and Nirwan Ansari^{ID}, *Fellow, IEEE*

Abstract—Fog computing is an emerging paradigm in provisioning computing and storage resources for the Internet-of-Things (IoT) devices. In a fog computing system, all devices can offload their data or computationally intensive tasks to nearby fog nodes, instead of to the distant cloud. As compared with cloud computing, fog computing can significantly reduce the transmission delay between IoT devices and computing servers. However, the current fog system is rather susceptible to malicious attacks. To increase the security level, we propose to partition the fog system into fog node clusters (FNCs), with fog nodes (FNs) in one cluster sharing the same access control list (ACL) which is protected by a blockchain. Generating blockchains requires tremendous computing power and can rapidly drain the computing capacities of FNs. In this article, we first customize the blockchain for FNC to reduce the required computing power consumption and storage spaces. Second, a new scheme is designed for the blockchain-based FNC (BFNC) to recover ACL automatically. In addition, we propose a heuristic algorithm to reduce the time to acquire hash values of blocks by computing cooperatively with all available devices. The simulation results have demonstrated that using the cooperative computing strategy can reduce the time of computing a block hash than noncooperative strategies.

Index Terms—Blockchain, fog computing, fog node clusters (FNCs), Internet of Things (IoT).

I. INTRODUCTION

IN THE era of the Internet of Things (IoT), different kinds of IoT devices [especially mobile devices (MDs)] are becoming indispensable parts of our daily life [1], [2]. However, MDs inherently consume tremendous computing power to facilitate various functions; this is against the requirements for high-performance devices, such as lightweight and long battery life [3]. Cloud computing has thus been developed to mitigate this limitation as MDs may offload computationally intensive tasks to cloud servers, which then return the results to MDs. Nevertheless, communicating with the remotely located cloud servers incurs a long transmission delay, and thus this paradigm is inherently prohibitive for latency-sensitive mobile applications, such as high-quality video streaming or real-time data analysis.

Fog computing was later proposed in 2012 by Cisco [4] as a solution to mitigate the long transmission delay in cloud

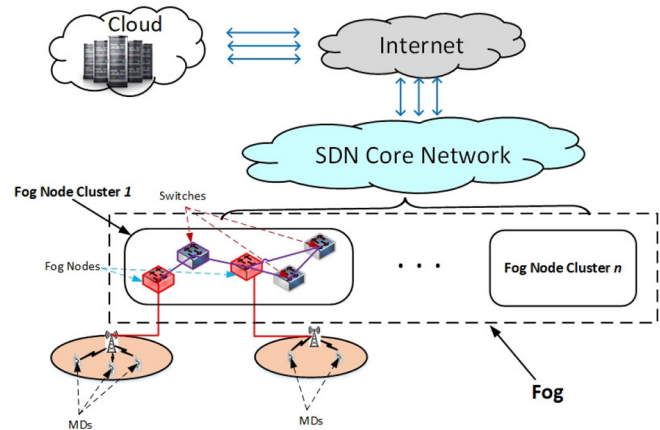


Fig. 1. Schematic illustration of the key components in the communication system: the cloud computing system, the Internet, and the SCN.

computing. As illustrated in Fig. 1, the fog computing system comprises several fog nodes (FNs) located at the edge of the software-defined network (SDN) core network. Fog computing refers to the architecture of having FNs and MDs at the edge to provision a substantial amount of computing power and storage spaces. The SDN core network (SCN) allows direct intercommunications among FNs [5]. FNs can be grouped into several FN clusters (FNCs) to provide efficient storage sharing within each FNC [6]. Each FNC can provide computing power for MDs within its coverage [7]. As compared with cloud computing, a smaller transmission delay can be achieved by FNs, as they are much closer to MDs. In addition, the architecture of fog computing enables flexible management of resources [8], [9].

FNs are more susceptible to malicious attacks than cloud servers [10], as the access control list (ACL) is deployed in FNs to reduce the transmission delay [11]. Identifying malicious users is an efficient method to secure FNs. Abbasi and Shah [12] combined user behavior profiling with the decoy technique to detect malicious behaviors. In terms of small-scale FNCs, which cannot afford a computationally intensive security strategy, Arya and Dave [13] proposed a security-based service broker policy by using a dynamic reconfiguration to monitor the characteristics of FNs. Small-scale networks have limited computing resources, and thus using complicated security schemes does incur long computing delays. To relieve the computing delay incurred by complex security methods, Kumar *et al.* [14] proposed the delimited anti-jammer scheme based on machine learning to secure the

Manuscript received October 11, 2019; revised November 8, 2019; accepted February 12, 2020. Date of publication February 17, 2020; date of current version July 10, 2020. (Corresponding author: Di Wu.)

The authors are with the Advanced Networking Laboratory, Department of Electrical Computer Engineering, New Jersey Institute of Technology, Newark, NJ 07102 USA (e-mail: dw245@njit.edu; nirwan.ansari@njit.edu). Digital Object Identifier 10.1109/JIOT.2020.2974231

network and alleviate the traffic congestion simultaneously. To increase the security level and reduce the complexity of encryption methods, Liu *et al.* [15] proposed the efficiency certificate-based signature scheme for wireless cooperative networks. Verma *et al.* [16] proposed the pairing free and provable certificate-based aggregate signature scheme, a computationally efficient method, to secure e-healthcare monitoring systems. The aforementioned works reduce the complexity of security schemes in different aspects. However, the computational complexities are still too high for small-scale FNCs.

In order to prevent unauthorized access to the network, the blockchain technology was adopted to ensure the synchronization and the accuracy of ACLs in one FNC [17]. Blockchain was first introduced by Satoshi Nakamoto in 2009 [18] to safeguard electronic cash systems. By dividing a fog system into several small-scaled P2P networks (i.e., FNCs), the required computing power for generating blockchains in each FNC is reduced (much smaller than that of bitcoin). It is highly promising to adopt the blockchain in FNCs, as the required computing power is significantly lowered [19]. However, as compared with the relatively lower computing power requirements, the computing time of acquiring a new block hash (BH) by FNs is still too long for a fog system.¹ In a blockchain-based FNC (BFNC), FNs provide the computing power to calculate the BH, while the blockchain safeguards the security of the BFNC. Thus, the computing time of BH needs to be further reduced in employing blockchains to secure BFNCs.

In this article, we propose the architecture of BFNCs. We design a mechanism for BFNCs to manage computing power. Considering that not all MDs of IoT have enough computing power, MDs with sufficient computing power help calculate BHs. To minimize the computing time of BHs, we propose a new resource allocation algorithm to appropriately manage computing power. The remainder of this article is organized as follows. In Section I, we introduce the architecture and mechanism of BFNCs. We formulate the problem of computing power allocation in BFNCs in Section II. The mathematical analysis of the problem is presented in Section III. A heuristic algorithm, referred to as time-aware computing set allocation algorithm (WEAR), is introduced to allocate computing power to calculate blocks' hash values in Section IV. The simulation results to demonstrate the performance of the proposed architecture are presented in Section V. A brief conclusion is drawn in Section VI.

II. BLOCKCHAIN-BASED FOG NODE CLUSTER

Most of the applications of blockchains (e.g., bitcoin) consume a large amount of computing power and require huge storage spaces. To reduce the computing power requirement and storage space requirement, we propose a BFNC. In a BFNC, the length of a blockchain is limited to a static length to save storage spaces. In addition, generating blockchains in BFNC, which is a small-scale P2P network, requires less computing power than in a large-scale P2P network.

¹The primary function of FNs is to help MDs with their offloaded tasks, which may occupy much of the computing power of FNs. Thus, FNs may not have enough computing power to calculate the BH of each block.

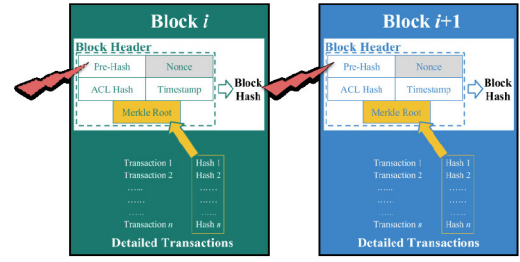


Fig. 2. Details of blocks in BFNC.

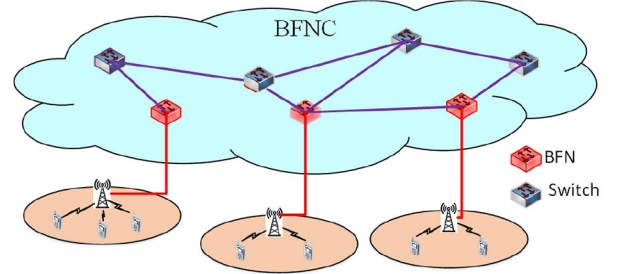


Fig. 3. Fog computing with blockchain.

A. Blockchain in BFNC

In one BFNC, the FNs that store the blockchain and the ACL are called blockchain-based FNs (BFNs). Each block records all decision results of MDs' requests (rejection or approval to access the network) and all changes of the ACL in the previous static timeslot [20], as illustrated in Fig. 2. Each request from an MD (including its returned result) or each update to the ACL is viewed as one transaction in the blockchain. The ACL Hash is the hash value of the ACL at the end of each timeslot, and it can be used to synchronize all ACLs stored in different BFNs (in one BFNC) at the beginning of the next timeslot.

Considering the limited storage space of each BFN in a BFNC, the length of the blockchain is constrained to a static number of blocks (i.e., old blocks are automatically deleted when the length reaches the upper bound) [21]. Unlike miners in bitcoin, BFNs in a BFNC are not rewarded after they finish calculating BHs. Thus, the relationships among BFNs are cooperative, rather than competitive. That is, all BFNs in one BFNC share their computing power and try to use the least time to acquire the first BH that satisfies the system requirements.

B. Architecture of BFNCs

In the architecture of a BFNC, as illustrated in Fig. 3, there exist two kinds of switches: 1) "traditional" switches, which are responsible for forwarding data only and 2) BDNs, which are equipped with computing power and storage spaces. Switches and BFNs are reachable from each other. ACLs and blockchains are stored in BFNs. Each BFN is connected to an access point. MDs can not only offload tasks to BFNs but also provide some computing power for calculating BHs.

A BFNC is a cluster of BFNs at the edge of an SCN. The central controller of an SCN only transmits necessary information to BFNCs (e.g., updating ACLs). Thus, a BFNC can be viewed as a P2P network.

III. PROBLEM FORMULATION AND ANALYSIS

A timeslot is defined as the computing time of the BH for a new block.² Thus, the faster a BFNC can acquire the BH of the current block, the sooner this BFNC can synchronize the ACLs stored in BFNs.³ Since a BFNC has limited computing power for computing BHs, the computing power should be appropriately allocated. We denote set Y as the set of acceptable BHs (i.e., a subset of all possible BHs).⁴ In different timeslots, the sets of acceptable BHs are different. The essence of computing the BH is to try different nonces in the hash function to get a hash value that falls in set Y . The computing set is defined as a set of nonces that are used as the inputs of a hash function by a miner⁵ to acquire hash values (i.e., BHs). For example, in SHA256 (Cryptographic Hash Algorithm 256), the whole computing set (denoted by $\mathcal{N}$) is a set that includes all integers from 0 to $2^{256} - 1$ (i.e., $\mathcal{N} = \{0, 1, \dots, 2^{256} - 1\}$). By allocating different sizes of computing sets to different BFNs, the computing time of acquiring the first BH within set Y varies.⁶

A. Problem Formulation

Let M be the set of BFNs in the BFNC. The total number of BFNs is $m = |M|$. Denote A_i as the set of MDs under the coverage of BFN i , $i \in M$. MDs in set A_i can only communicate with BFN i .

Instead of having all MDs and BFNs trying nonce randomly in $\mathcal{N}$, we divide $\mathcal{N}$ into several subsets. By allocating these subsets to different devices (including MDs and BFNs), a BFNC can reduce the computing time of acquiring the first acceptable BH. The computing set allocated to each BFN will be divided into several subsets and allocated to the MDs within its coverage (the subset allocated to MD j under the coverage of BFN i is denoted by N_{ij}). The size of set N_{ij} is denoted by n_{ij} . Set $\mathbf{n}$ is used to denote the size of computing sets that allocated to all MDs and BFNs, i.e., $\mathbf{n} = \{n_{ij} | i \in M, j \in A_i\}$. T_{ij} is the computing time of MD j under the coverage of BFN i in acquiring the first acceptable BH. D_{ij} is the network delay incurred for MD j to transmit a BH to BFN i . Thus, $T_{ij} + D_{ij}$ is the total time of computing and transmitting a BH from MD j to BFN i . We define a new set that includes both BFN i and the MDs within the coverage of BFN i as $A'_i = \text{BFN } i \cup A_i \forall i \in M$.

We formulate the problem of minimizing the computing time of the first acceptable BH as

$$\begin{aligned} \mathbf{P0} : & \min_{\mathbf{n}} \min_{i \in M, j \in A'_i} T_{ij} + D_{ij} \\ \text{s.t.: } & C1 : H_{ij} \in Y \quad \forall j \in A'_i, i \in M \\ & C2 : \prod_{k=1}^{r-1} (H^k - H_{ij}) \neq 0 \quad \forall j \in A'_i, i \in M \end{aligned}$$

²In each timeslot, all BFNs keep recording the transactions for the current timeslot and calculating BHs for the previous timeslot, simultaneously.

³Once a BFNC synchronizes the ACLs of its BFNs, the malicious users need to restart the hacking process to control their target BFNs and get the necessary information (e.g., the ACL). Thus, the more frequently a BFNC synchronizes the ACLs of its BFNs, the more secure it is.

⁴The acceptable BHs are defined as the BHs that falls in set Y .

⁵A miner is a device with computing power for calculating the hash values. In a BFNC, both BFNs and MDs can be miners.

⁶If the acceptable BH is calculated by an MD, it takes an additional transmission delay and processing delay before the BFN gets the result. Thus, we denote these delays as network delay.

$$C3 : \bigcup_{j \in A'_i, i \in M} N_{ij} = \mathcal{N}.$$

Constraint C1 requires the BHs, calculated by MDs and BFNs, to be acceptable BHs (i.e., the BHs $\in Y$). H_{ij} is the BH calculated by MD j under the coverage of BFN i (the time consumption is $T_{ij} + D_{ij}$). Constraint C2 ensures the new acceptable BH H_{ij} to be unique in this blockchain (i.e., the value of H_{ij} is different from the BHs' value of all other blocks in the blockchain). We define r as the length of stored blockchain.⁷ H^k is the BH of block k ($k \leq r$) stored in the blockchain. Constraint C3 ensures the union of subsets N_{ij} covers the computing set $\mathcal{N}$.

B. Problem Analysis

The hash function is not an invertible function. The only way to determine the input for a given output is to construct a table of enumerating every input and its corresponding output. Constructing the table requires to take the hash function on all inputs and to record all corresponding results. In the scenario of a blockchain with a huge set of inputs, constructing such a huge mapping table is impossible and impractical.⁸ Thus, to get the input (i.e., nonce) when the output (BH) is within set Y , there is no other way than trying different nonces. In hash functions, each input has the same probability of generating output within the set Y . To make the analysis tractable, the average computing time is used to characterize the amount of time on average to generate the first BH in set Y . Thus, in $\mathbf{P0}$, the time of acquiring the first acceptable BH is replaced by the average computing time of acquiring the first acceptable BH (i.e., $\bar{T}_{ij}$ is used to replace T_{ij}). $\bar{T}_{ij}$ is the average computing time of acquiring the first acceptable BH by MD j within the coverage of BFN i .

As compared with the large input set ($\mathcal{N}$), the number of occupied hash values $r - 1$ (i.e., the number of blocks) is negligible. Thus, ρ is the ratio of acceptable BHs with respect to d and $\mathcal{N}$ as $\rho = d/|\mathcal{N}|$. The set of inputs ($\mathcal{N}$) is divided into several subsets, and these subsets are assigned to different BFNs as their respective sets of inputs (i.e., the computing set). For MD j under the coverage of BFN i (the computing set is N_{ij}), the probability of generating the first acceptable BH at the x th trial of nonce is $\rho(1 - \rho)^{x-1}$. The average number of trials in acquiring the first acceptable BH by MD j within the coverage of BFN i (E_{ij}) is

$$E_{ij} = \sum_{x=1}^{n_{ij}} x \cdot \rho \cdot (1 - \rho)^{x-1}. \quad (1)$$

$E_{ij} = 1/\rho$ when $n_{ij} \rightarrow \infty$. Here, we make an approximation that $E_{ij} = 1/\rho$ when $E_{ij} > 0.99 \cdot 1/\rho$. By substituting $E_{ij} \geq 0.99 \cdot 1/\rho$ into (1), we make the approximation of n_{ij} as

$$n_{ij} \geq 10/\rho. \quad (2)$$

⁷We assume that the blockchains have already reached the maximum length in the problem formulation.

⁸If malicious users want to acquire this mapping table, they need to know the exact hash function between the input and the output. If given enough time, malicious users can get the mapping table and hack target users. In realistic applications of the hash function, the users change the hash function regularly to keep their data secured.

Thus, the probability of having at least one acceptable BH with the computing set N_{ij} (P_{ij}) is

$$P_{ij} = \sum_{x \in N_{ij}} \rho \cdot (1 - \rho)^{x-1} = 1 - (1 - \rho)^{n_{ij}}. \quad (3)$$

By plugging inequality (2) into (3), we have

$$P_{ij} \geq 1 - (1 - \rho)^{10/\rho}. \quad (4)$$

Since $1 - (1 - \rho)^{10/\rho} \geq 0.99$ is true for any $\rho \in (0, 1)$, we can deduce that $P_{ij} \geq 0.99$ is always true. Thus, if n_{ij} satisfies (2), at least one acceptable BH surely exists when the computing set is N_{ij} .

Thus, there exists at least one acceptable BH when the size of the computing set is larger than $10/\rho$. In this problem, we limit the size of the computing set for each BFN to $10/\rho$ (i.e., $n_i = 10/\rho \forall i \in \mathbf{M}$). Hence, the computing set of any MD is smaller than $10/\rho$, i.e.,

$$n_{ij} < 10/\rho \quad \forall j \in A'_i \quad \forall i \in \mathbf{M}. \quad (5)$$

Thus, for all $j \in A'_i$, we always have $n_{ij} < n_i < 10/\rho$. E'_{ij} is the average number of trials by MD j under the coverage of BFN i in acquiring the first acceptable BH (if an MD cannot find any acceptable BH after trying all nonces in its computing set, the number of trials is equal to the size of its computing set). E'_{ij} is acquired by

$$E'_{ij} = E_{ij} \cdot [1 - (1 - \rho)^{n_{ij}}] + n_{ij} \cdot (1 - \rho)^{n_{ij}} \quad (6)$$

where E_{ij} is the average number of trials in acquiring the first acceptable BH if it exists within the computing set N_{ij} for MD j under the coverage of BFN i [see (1)]. We denote the set of BHs corresponding to the computing set N_{ij} as $\mathbf{H}_{ij}$. Note that $(1 - \rho)^{n_{ij}}$ is the probability that none of the BHs in $\mathbf{H}_{ij}$ is acceptable. Thus, the average computing time of acquiring the first acceptable BH by MD j under the coverage of BFN i ($\bar{T}_{ij}$), including the network delay, is

$$\bar{T}_{ij} = E'_{ij}/c_{ij} + D_{ij} \quad (7)$$

where D_{ij} is the network delay between MD j and BFN i . c_{ij} is the computing power⁹ of MD j under the coverage of BFN i .

In a BFNC, both MDs and BFNs may be hacked and become malicious users/devices, which do not help calculate BHs. We assume that BFNs cannot tell which MDs are malicious. Thus, the computing sets allocated to malicious MDs are wasted. Hence, the average number of trials of a malicious MD in acquiring an acceptable BH is equal to the size of its computing set n_{ij} . By considering the network delay, the computing time of a malicious MD to acquire an acceptable BH is

$$T_{ij}^m = n_{ij}/c_{ij} + D_{ij}. \quad (8)$$

We introduce a parameter b_{ij} to denote the probability of normal MDs. Thus, we denote the average computing time of acquiring the first acceptable BH by an MD, which has the probability $(1 - b_{ij})$ of being malicious, by $\hat{T}_{ij}$

$$\hat{T}_{ij} = b_{ij} \cdot \bar{T}_{ij} + (1 - b_{ij}) \cdot T_{ij}^m. \quad (9)$$

In a BFN with computing set $|N_i| = 10/\rho$, the average time of acquiring the first acceptable BH is equivalent to the average time of all MDs (including the BFN) in completing their

computing.¹⁰ Thus, the computing time of acquiring the first acceptable BH is transformed from $\mathbf{P0}$ to $\mathbf{P1}$ (i.e., minimizing the average computing time of acquiring the first acceptable BH for each BFN) as

$$\mathbf{P1} : \arg \min_{\mathbf{n}_i} : \sum_{j \in A'_i} \hat{T}_{ij}/|A'_i|$$

$$\text{s.t.: } C4 : \left| \bigcup_{j \in A'_i} N_{ij} \right| = \frac{10}{\rho}$$

$$C5 : N_{ij} \cap N_{ij'} = \emptyset \quad \forall j, j' \in A'_i, j \neq j'.$$

$\mathbf{n}_i$ is the set defined as $\mathbf{n}_i = \{n_{ij} | j \in A'_i\}$. The computing sets are allocated to MDs as depicted in Constraint C4. Constraint C5 ensures the computing sets not to overlap with each other.

IV. TIME-AWARE COMPUTING SET ALLOCATION ALGORITHM

We propose a heuristic algorithm, i.e., time WEAR, to efficiently solve $\mathbf{P1}$. The basic idea of WEAR, summarized in Algorithm 1, is to decompose problem $\mathbf{P1}$ into two parts, i.e., computing sets initialization and computing sets adjustment.

Computing Set Initialization: The MDs that can be used to help BFN i calculate the BH are denoted by A'_i (the BFN i is included in A'_i). Get $\hat{T}_{ij}$ from (9) with $n_{ij} = 10/(\rho \cdot |A'_i|) \forall j \in A'_i$. The set $\mathbf{T}_i$ stores $\hat{T}_{ij}$ in descending order.

Computing Set Adjustment: All the available MDs are assigned with the initial size of computing sets after the computing set initialization. $\hat{T}_{ij}$ varies due to different available computing power of MDs and the network delays between MDs and BFN i . The MDs' initial computing sets need to be adjusted to reduce the longest computing time among all MDs within the coverage of BFN i (i.e., by reducing the size of the computing set with large $\hat{T}_{ij}$ and increasing the size of other MDs' computing sets). Specifically, we have the following steps.

- 1) Sort MDs' available computing power (c_{ij}) in descending order and stored in set $\mathbf{C}_i$. Reduce the computing set of the MD with the largest $\hat{T}_{ij}$ and correspondingly increase the computing set of the MD with the largest c_{ij} .
- 2) We label the index j of the largest $\hat{T}_{ij}$ in $\mathbf{T}_i$ as j'' and label the index j of the largest c_{ij} in $\mathbf{C}_i$ as j' . Reduce $n_{ij''}$ and increase $n_{ij'}$ until any of the following conditions are met:
 - a) $n_{ij''} = 0$;
 - b) $\hat{T}_{ij''} = \hat{T}_{ij'}^+$. $\hat{T}_{ij'}^+$ is defined as $\hat{T}_{ij'}^+ = \min\{\hat{T}_{ij} \in \mathbf{T}_i : j \in A'_i, \hat{T}_{ij} > \hat{T}_{ij'}\}$. If the reduction of $n_{ij''}$ is terminated by $\hat{T}_{ij''} = \hat{T}_{ij'}^+$, we mark MD j'' as a temporarily unavailable MD,¹¹ and remove $c_{ij''}$ from $\mathbf{C}_i$.
- 3) Repeat steps 1) and 2) iteratively until any of the following conditions is met:

¹⁰One MD completes its computing in either one of two cases: the MD has tried all nonces in their computing set but does not get an acceptable BH or the MD get an acceptable BH.

¹¹If the computing set of an MD cannot be change in an iteration of the computing set adjustment, this MD is considered as a temporarily unavailable MD. The temporarily unavailable MDs are placed in set U_i .

⁹The computing power is defined as the number of nonces that a device can try per second.

Algorithm 1 WEAR Algorithm

```

1: Get the set of available MDs  $A'_i$ .
2: Equally allocate computing set as  $n_{ij} = 10/(\rho \cdot |A'_i|)$ ,  $\forall j \in A'_i$ 
3: Get  $\hat{T}_{ij}$ ,  $\forall j \in |A'_i|$  by (9)
4: Sort  $\hat{T}_{ij}$  in descending order and store in  $T_i$ 
5: Sort  $c_{ij}$  in descending order and store in  $C_i$ 
6: while  $|U_i| < |A'_i|$  or  $j'' = j'$  do
7:   Label index  $j$  of the first  $\hat{T}_{ij}$  in  $T_i$  as  $j''$ 
8:   Label index  $j$  of the first  $c_{ij}$  in  $C_i$  as  $j'$ 
9:   Get new  $\hat{T}'_{ij'}$  with  $n_{ij'} = n_{ij'} + n_{ij''}$  by (9)
10:  if  $\hat{T}'_{ij'} > \hat{T}^+_{ij'}$  then
11:    Acquire  $n'_{ij'}$  with  $\hat{T}'_{ij'} = \hat{T}^+_{ij'}$ 
12:    Update  $n_{ij'}$  by  $n_{ij'} = n_{ij'} + n_{ij''} - n'_{ij'}$ 
13:    Put  $j''$  in set  $U_i$ 
14:  else
15:    Set  $n_{ij''} = 0$ ,  $\hat{T}_{ij''} = \hat{T}'_{ij'}$ 
16:  end if
17: end while
18: if  $\sum n_{ij} < 10/\rho$  then
19:   Set  $U_i = \phi$  and go to step 4
20: end if

```

- a) all MDs are marked as “unavailable MDs”;
- b) $j'' = j'$.

V. SIMULATION RESULTS

In this simulation, we minimize the computing time of acquiring the first acceptable BH based on a BFN with 20 MDs. Thus, we ignore the BFN index i in the following analysis. The available computing power (c_j) of MDs follows a normal distribution $N(500, 200)$ (hash/s). The network delay D_j follows a normal distribution $N(2000, 1000)$ (milliseconds). The probability of having acceptable BHs is $\rho = 10^{-4}$.

The performance of WEAR is compared with two existing allocation strategies, i.e., the equal division allocation (EDA) and the ratio-oriented allocation (RDA). EDA divides the computing set equally and assigns them to all devices with computing power. During the computing set allocation, EDA does not consider the available computing power and the network delay. RDA allocates the computing sets based on MDs' available computing power, i.e., the MD with a larger computing power gets a larger computing set [22].

The probability of having acceptable BHs (ρ) has a great influence on the computing time of acquiring the first acceptable BH. Fig. 4 shows the relationship between ρ and the computing time of WEAR. As the probability of acceptable BHs increases, the computing time of acquiring the first acceptable BH by WEAR reduces.

The computing time with different numbers of MDs under the coverage of one BFN is shown in Fig. 5. Having more MDs in the coverage of one BFN can increase the total available computing power, thus incurring a shorter computing time of acquiring the first acceptable BH for all three algorithms. This is because as the number of MDs increases, the computing set allocated to each MD is becoming smaller, and thus

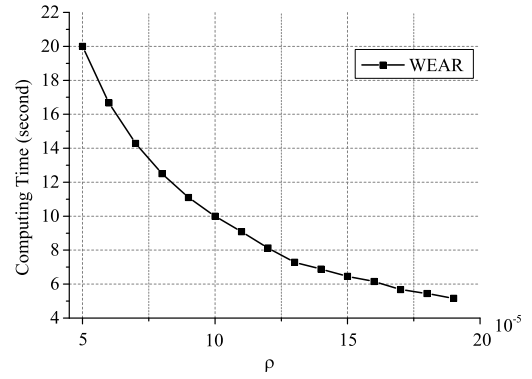
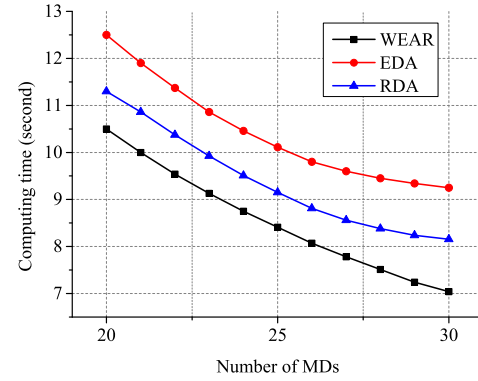
Fig. 4. Computing time with different ρ .

Fig. 5. Computing time with different numbers of MDs.

it takes each MD less time to search for an acceptable BH in its associated computing set. Hence, the network delay is contributing a larger weight in the total computing time. WEAR can jointly consider the network delay and the size of computing sets in making the computing set allocation to reduce the computing time of acquiring the first acceptable BH, while EDA and RDA cannot.

Another parameter that influences the computing time of acquiring the first acceptable BH is the available computing power of MDs. The computing time of acquiring the first acceptable BH by different algorithms with different average available computing power is shown in Fig. 6. The computing time of acquiring the first acceptable BH by WEAR is smaller than that by RDA and EDA with the same available computing power because WEAR jointly considers the available computing power and the network delay in making the computing set allocation. RDA allocates large computing sets to MDs with large computing power, and can efficiently decrease the computing time better than EDA does.

The computing time of acquiring the first acceptable BH with a different standard deviation of MDs' available computing power is shown in Fig. 7. RDA and WEAR allocate computing sets by taking into consideration of available computing power of devices. Thus, RDA and WEAR allocate larger computing sets to devices with larger computing power. Different from RDA and WEAR, EDA equally allocates computing sets to MDs. Thus, the computing time of acquiring the first acceptable BH by EDA is decided by the MDs with the smallest computing set. As the standard deviation increases,

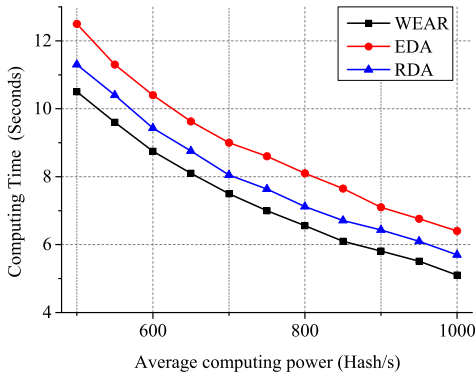


Fig. 6. Computing time with different average computing power.

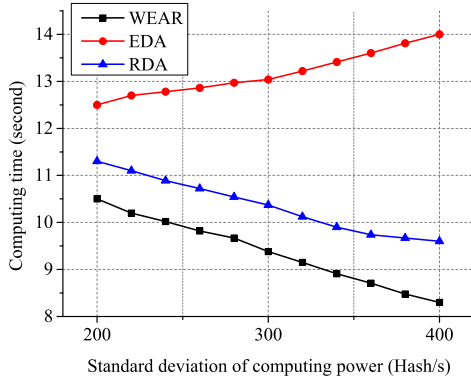


Fig. 7. Computing time of acquiring acceptable BHs with different standard deviations of computing power.

the probability of MDs with small computing power increases. Thus, the computing time of acquiring the first acceptable BH by EDA is increasing with the standard deviation of computing power.

The network delay between the BFN and MDs can influence the computing time of acquiring the first acceptable BH. The impact of the network delay on the computing time of acquiring the first acceptable BH is shown in Figs. 8 and 9. EDA and RDA do not consider the network delay in computing set allocations, and thus the standard deviation of the network delay has little influence on their computing times, as shown in Fig. 8. By adjusting the size of computing sets according to network delays, WEAR achieves shorter computing time than EDA and RDA. WEAR can achieve shorter computing time with a larger network delay range. Unlike the standard deviation, the average value of the network delay impacts all three algorithms, as shown in Fig. 9. As the average network delay increases, the computing time of EDA, RDA, and WEAR increases correspondingly. With the consideration of the network delay in computing sets allocation by WEAR, the increase of the average network delay has less influence on WEAR.

The appearance of malicious MDs incurs a different computing set allocation and thus influences the computing time of acquiring the first acceptable BH by all three algorithms. In the simulation, malicious MDs are selected with a probability

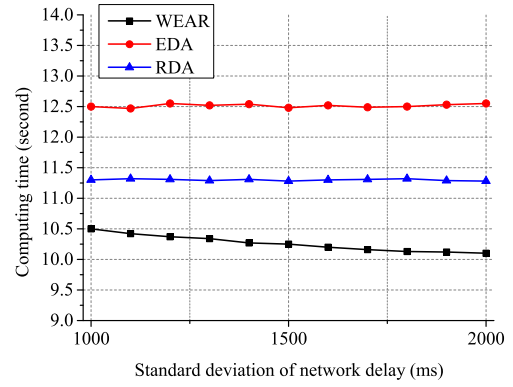


Fig. 8. Computing time of acquiring acceptable BHs with different standard deviation network delay.

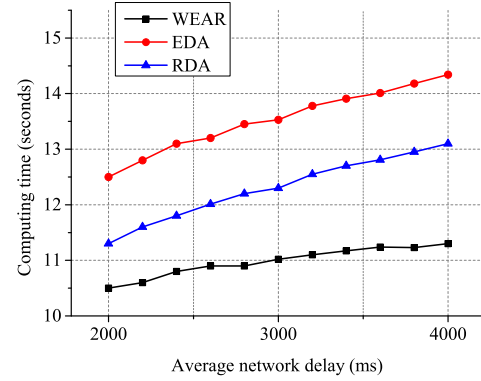


Fig. 9. Computing time of acquiring acceptable BHs with different average network delay.

related to their available computing powers.¹² The computing time of acquiring acceptable BHs by different algorithms with different numbers of malicious MDs is illustrated in Fig. 10.

With a small number of malicious MDs in a BFN, the computing time of acquiring the first acceptable BH of all the three algorithms do not increase too much because MDs with lower available computing power have a higher probability of being malicious. In addition, the MDs with small computing power are allocated with small computing sets by WEAR and RDA. Thus, the computing time of acquiring the first acceptable BH by WEAR and RDA is shorter than that by EDA. As the number of malicious MDs increases, more computing sets are allocated to malicious MDs. The available computing power of normal MDs becomes smaller. Thus, the computing time of acquiring the first acceptable BH of all three algorithms increases sharply with a large number of malicious MDs. Since RDA and EDA do not consider the network delay from MDs to the BFN in computing set allocation, the computing time by RDA and EDA increases faster than that of WEAR.

VI. CONCLUSION

In this article, we have proposed the architecture of BFNC and designed the WEAR algorithm to reduce the time of

¹²Once an MD is hacked by other devices, part of its computing power will be taken to receive control information from the hacker. Thus, we assume that MDs with less available computing power have a higher probability of being hacked [23].

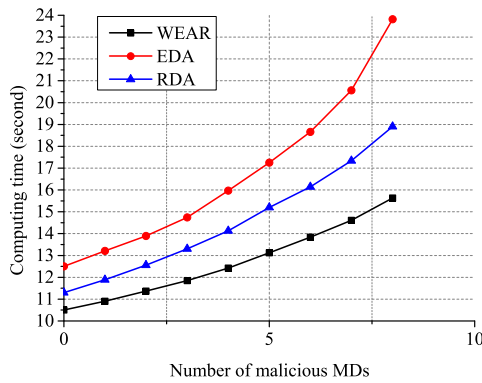
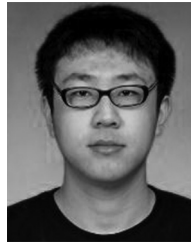


Fig. 10. Computing time of acquiring the first acceptable BH with different number of malicious MDs.

computing the hash values of blocks. In comparison with two algorithms (EDA and RDA), WEAR can leverage MDs with largely available computing powers. Even with malicious MDs, WEAR still achieves the shortest computing time of acquiring the first acceptable BH than EDA and RDA. Thus, we can conclude that among the three algorithms, WEAR can achieve the highest security level by quickly acquiring acceptable BHs in different security challenging environments.

REFERENCES

- [1] X. Sun and N. Ansari, "EdgeIoT: Mobile edge computing for the Internet of Things," *IEEE Commun. Mag.*, vol. 54, no. 12, pp. 22–29, Dec. 2016.
- [2] N. Ansari and X. Sun, "Mobile edge computing empowers Internet of Things," *IEICE Trans. Commun.*, vol. 101, no. 3, pp. 604–619, 2018.
- [3] R. Krejčí, O. Hujňák, and M. Švepeš, "Security survey of the IoT wireless protocols," in *Proc. 25th Telecommun. Forum (TELFOR)*, Nov. 2017, pp. 1–4.
- [4] F. Bonomi, R. Milito, J. Zhu, and S. Addepalli, "Fog computing and its role in the Internet of Things," in *Proc. 1st Ed. MCC Workshop Mobile Cloud Comput.*, 2012, pp. 13–16.
- [5] A. Basta, A. Blenk, K. Hoffmann, H. J. Morper, M. Hoffmann, and W. Kellerer, "Towards a cost optimal design for a 5G mobile core network based on SDN and NFV," *IEEE Trans. Netw. Service Manag.*, vol. 14, no. 4, pp. 1061–1075, Dec. 2017.
- [6] Y. Xiao and M. Krunz, "QoE and power efficiency tradeoff for fog computing networks with fog node cooperation," in *Proc. IEEE INFOCOM IEEE Conf. Comput. Commun.*, May 2017, pp. 1–9.
- [7] X. Huang and N. Ansari, "Secure multi-party data communications in cloud augmented IoT environment," in *Proc. IEEE Int. Conf. Commun. (ICC)*, May 2017, pp. 1–6.
- [8] M. Mukherjee, L. Shu, and D. Wang, "Survey of fog computing: Fundamental, network applications, and research challenges," *IEEE Commun. Surveys Tuts.*, vol. 20, no. 3, pp. 1826–1857, 3rd Quart., 2018.
- [9] Q. Fan and N. Ansari, "Towards workload balancing in fog computing empowered IoT," *IEEE Trans. Netw. Sci. Eng.*, early access, doi: 10.1109/TNSE.2018.2852762.
- [10] J. Kang, O. Simeone, J. Kang, and S. Shamai Shitz, "Control-data separation with decentralized edge control in fog-assisted uplink communications," *IEEE Trans. Wireless Commun.*, vol. 17, no. 6, pp. 3686–3696, Jun. 2018.
- [11] I. Riabi, L. A. Saidane, and H. K.-B. Ayed, "A proposal of a distributed access control over fog computing: The ITS use case," in *Proc. Int. Conf. Perform. Eval. Model. Wired Wireless Netw. (PEMWN)*, Nov. 2017, pp. 1–7.
- [12] B. Z. Abbasi and M. A. Shah, "Fog computing: Security issues, solutions and robust practices," in *Proc. 23rd Int. Conf. Autom. Comput. (ICAC)*, Sep. 2017, pp. 1–6.
- [13] D. Arya and M. Dave, "Security-based service broker policy for fog computing environment," in *Proc. 8th Int. Conf. Comput. Commun. Netw. Technol. (ICCCNT)*, Jul. 2017, pp. 1–6.
- [14] S. Kumar, K. Singh, S. Kumar, O. Kaiwartya, Y. Cao, and H. Zhou, "Delimited anti jammer scheme for Internet of Vehicle: Machine learning based security approach," *IEEE Access*, vol. 7, pp. 113311–113323, 2019.
- [15] J. K. Liu, F. Bao, and J. Zhou, "Short and efficient certificate-based signature," in *Proc. Int. Conf. Res. Netw.*, Heidelberg, Germany, 2011, pp. 167–178.
- [16] G. K. Verma, B. B. Singh, N. Kumar, O. Kaiwartya, and M. S. Obaidat, "Pfcbas: Pairing free and provable certificate-based aggregate signature scheme for the e-healthcare monitoring system," *IEEE Syst. J.*, early access, doi: 10.1109/JSYST.2019.2921788.
- [17] P. K. Sharma, M.-Y. Chen, and J. H. Park, "A software defined fog node based distributed blockchain cloud architecture for IoT," *IEEE Access*, vol. 6, pp. 115–124, 2018.
- [18] S. Nakamoto. (2009). *Bitcoin: A Peer-to-Peer Electronic Cash System*. [Online]. Available: <https://bitcoin.org/bitcoin.pdf>
- [19] A. Stanciu, "Blockchain based distributed control system for edge computing," in *Proc. 21st Int. Conf. Control Syst. Comput. Sci. (CSCS)*, May 2017, pp. 667–671.
- [20] O. Novo, "Blockchain meets IoT: An architecture for scalable access management in IoT," *IEEE Internet Things J.*, vol. 5, no. 2, pp. 1184–1195, Apr. 2018.
- [21] M. Dai, S. Zhang, H. Wang, and S. Jin, "A low storage room requirement framework for distributed ledger in blockchain," *IEEE Access*, vol. 6, pp. 22970–22975, 2018.
- [22] L. Fawcett, M. Broadbent, and N. Race, "Combinatorial auction-based resource allocation in the fog," in *Proc. 50th Eur. Workshop Softw. Defined Netw. (EWSN)*, Oct. 2016, pp. 62–67.
- [23] D. Zhou and H. Inamura, "Towards trustworthy and secure kiosk computing for mobile users," in *Proc. IEEE/IFIP Int. Conf. Embedded Ubiquitous Comput.*, vol. 2, Dec. 2008, pp. 49–54.



Di Wu (Student Member, IEEE) received the B.E. degree in communication engineering and the M.E. degree in information and communication engineering from Harbin Institute of Technology, Harbin, China. He is currently pursuing the Ph.D. degree in electrical engineering with the New Jersey Institute of Technology (NJIT), Newark, NJ, USA.

His research interests include mobile edge computing, network security, and resource management in fog computing.



Nirwan Ansari (Fellow, IEEE) received the Ph.D. degree from Purdue University, West Lafayette, IN, USA, the M.S.E.E. degree from the University of Michigan, Ann Arbor, MI, USA, and the B.S.E.E. degree (*summa cum laude*) with a perfect GPA from the New Jersey Institute of Technology (NJIT), Newark, NJ, USA.

He is a Distinguished Professor of electrical and computer engineering at NJIT. He authored *Green Mobile Networks: A Networking Perspective* (Wiley-IEEE, 2017) with T. Han, and coauthored two other books.

He has also (co)authored more than 600 technical publications, with over 280 published in widely cited journals/magazines. He also been granted more than 40 U.S. patents. His current research focuses on green communications and networking, cloud computing, drone-assisted networking, and various aspects of broadband networks.

Dr. Ansari has guest edited a number of special issues covering various emerging topics in communications and networking. He has served on the editorial/advisory board of over ten journals including as an Associate Editor-in-Chief of the *IEEE Wireless Communications Magazine*. He was elected to serve in the IEEE Communications Society (ComSoc) Board of Governors as a Member-at-Large, has chaired some ComSoc technical and steering committees, has been serving in many committees such as the IEEE Fellow Committee, and has been actively organizing numerous IEEE International Conferences/Symposia/Workshops. He is frequently invited to deliver keynote addresses, distinguished lectures, tutorials, and invited talks. Some of his recognitions include several Excellence in Teaching awards, a few best paper awards, the NCE Excellence in Research Award, the ComSoc TC-CSR Distinguished Technical Achievement Award, the ComSoc AHSN TC Technical Recognition Award, the IEEE TCGCC Distinguished Technical Achievement Recognition Award, the NJ Inventors Hall of Fame Inventor of the Year Award, the Thomas Alva Edison Patent Award, Purdue University Outstanding Electrical and Computer Engineering Award, the NCE 100 Medal, and designation as a COMSOC Distinguished Lecturer.